455.24411ptTasks to be completed:

- Add functionality to compute Hessians for integrals defined on the the skeleton triangulation.
- Solve the thermal compliance problem using the CG-Steihaug method in Optim.jl using a penalty method for the volume constraint
- Experiment with constrained newton-krylov methods in SciPy
- Decide on the way to benchmark the methods against first order / non-eliptically preconditioned methods.

1. Introduction

Newton methods can often converge faster than gradient-based methods by exploiting second-order information. In this work, we investigate Newton-based methods for level set topology optimization. Although such methods have been studied in topology optimization, they have not seen widespread adoption.

For density based methods, improved convergence properties by using exact hessian information in an SQP algorithm has been shown [**RojasLabanda2016**,**RojasLabanda2017**]. The authors note that the solving the subproblem is expensive. They do, however show promise of the use of second order information for accelerated convergence in the non-convex optimization landscape of topology optimization problems. It is also not clear how to extend this method beyond compliance minimization problems.

For level set methods, newton-based methods are less studied. In the optimize then discretize setting, improved convergence properties have once again been shown by using full hessians on a few problems [**Allaire2016**] by considering normal boundary perturbations for simple topology optimisation problems. Promising results are also shown for basic shape recoveries in boundary fitted settings [**Gangl2020**,**Etling2018**] by using second order shape derivatives.

A well known issue with newton methods is that they can require many iterations to converge if the initial guess is not close to the solution. Homotopy methods have been proposed to mitigate this issue by gradually transforming a convex problem into the original problem, allowing the solution to be tracked along the way. This is used in a level-set context by [**Gangl2024**] and in a density context by [**Papadopoulos2021**]. These methods, however require problem specific tuning of the continuation parameters. The method that we hope to propose in this work is a reparametrisation of the design space which allows superlinear convergence in early iterations without the need for continuation.

2. optimisation

We propose here an inexact CG-newton method for solving the optimisation problem:

$$\min_p \hat{J}(p), \tag{1}$$

where

$$\hat{J}(p) = J(u(p), p). \tag{2}$$

Consider first the taylor series expansion of the objective $\hat{J}$ around a point p_k:

$$\hat{J}(p_k + \tilde{p}_k) = \hat{J}(p_k) + \frac{d\hat{J}}{dp}(p_k)^\top \tilde{p}_k + \frac{1}{2}\tilde{p}_k^\top H(p_k)\tilde{p}_k + \mathcal{O}(|\tilde{p}_k|^3). \tag{3}$$

Ignoring the $\mathcal{O}(|\tilde{p}_k|^3)$ terms, we find $\tilde{p}_k$ that minimises this quadratic expression by setting its gradient with respect to $\tilde{p}_k$ to zero to obtain the Newton equation:

$$H(p_k)\tilde{p}_k = -\frac{d\hat{J}}{dp}(p_k). \tag{4}$$

To avoid forming the full hessian matrix, we use the CG method to iteratively solve this system, which instead only requires the hessian-vector product $H(p_k)v$ for some vector v. We want to choose an appropriate forcing sequence η_k to terminate the CG iterations when the residual $r_k = H(p_k)\tilde{p}_k + \frac{d\hat{J}}{dp}(p_k)$ satisfies:

$$\|r_k\|_2 \leq \eta_k \left\| \frac{d\hat{J}}{dp}(p_k) \right\|_2. \tag{5}$$

We use the choice:

$$\eta_k = \min\left(0.5, \sqrt{\left\| \frac{d\hat{J}}{dp}(p_k) \right\|}\right). \tag{6}$$

We can prove (see e.g. [**NocedalWright2006**]) that choosing $\eta_k = O\left(\left\|\frac{d\hat{J}}{dp}(p_k)\right\|\right)$ leads to linear convergence for the iterations:

$$\left\| \frac{d\hat{J}}{dp}(p_{k+1}) \right\| \leq C \left\| \frac{d\hat{J}}{dp}(p_k) \right\|, \tag{7}$$

and that choosing $\eta_k = O\left(\left\|\frac{d\hat{J}}{dp}(p_k)\right\|^2\right)$ leads to quadratic (local) convergence:

$$\left\| \frac{d\hat{J}}{dp}(p_{k+1}) \right\| \leq C \left\| \frac{d\hat{J}}{dp}(p_k) \right\|^2. \tag{8}$$

albeit this tighter restriction may require too many CG iterations. The chosen η_k is a compromise between these two extremes. Since the matrix is not necessarily positive definite, we also terminate if a direction of negative curvature is found [**NocedalWright2006**].

$$\tilde{p}_k^T H(p_k)\tilde{p}_k < 0. \tag{9}$$

An adaptive trust region is also used. We aim to compute the Hessian action in a direction $\tilde{p}$ for an objective function $J(u, p)$ constrained by a partial differential equation $F(u, p) = 0$, where u is the state variable and p is the parameter we wish to optimize. The state variable u depends on the parameter p through the pde constraint.

2.1. **Forward Pass.** Our goal is to compute the objective:

$$J(u(p), p) \tag{10}$$

Firstly, we must solve the **state equation** for u given p:

$$F(u, p) = 0. \tag{11}$$

Them, we can substitute this into the objective function $J(u, p)$.

2.2. **Adjoint Pass.** Next, our goal is to compute the gradient $\frac{d}{dp}J(u(p),p)$. Using the chain rule, we have:

$$\frac{d}{dp}J(u(p),p) = \frac{\partial J}{\partial p}(u,p) + \frac{\partial J}{\partial u}(u,p)\frac{du}{dp}. \tag{12}$$

We can easily compute the partial derivatives $\frac{\partial J}{\partial p}(u,p)$ and $\frac{\partial J}{\partial u}(u,p)$, but we also need to compute the term $\frac{du}{dp}$. To obtain this, we rely on the implicit function theorem by differentiating the pde constraint $F(u,p)=0$ with respect to p:

$$\frac{\partial F}{\partial u}(u,p)\frac{du}{dp} + \frac{\partial F}{\partial p}(u,p) = 0, \tag{13}$$

which gives

$$\frac{du}{dp} = -\left(\frac{\partial F}{\partial u}(u,p)\right)^{-1}\frac{\partial F}{\partial p}(u,p). \tag{14}$$

Substituting this into the gradient expression, we have:

$$\frac{d}{dp}J(u(p),p) = \frac{\partial J}{\partial p}(u,p) - \frac{\partial J}{\partial u}(u,p)\left(\frac{\partial F}{\partial u}(u,p)\right)^{-1}\frac{\partial F}{\partial p}(u,p). \tag{15}$$

To avoid computing the inverse of $\frac{\partial F}{\partial u}(u,p)$ or solving a linear system for each component of the gradient, we introduce the adjoint variable λ defined as the solution to the **adjoint equation**:

$$\left(\frac{\partial F}{\partial u}(u,p)\right)^{T}\lambda = \left(\frac{\partial J}{\partial u}(u,p)\right)^{T}. \tag{16}$$

With this, we can express the gradient as:

$$\frac{d}{dp}J(u(p),p) = \frac{\partial J}{\partial p}(u,p) - \lambda^{T}\frac{\partial F}{\partial p}(u,p). \tag{17}$$

2.3. **Hessian action.** Finally, we want to compute the Hessian action $H(p)(\tilde{p})$. We start by differentiating the gradient expression with respect to p in the direction $\tilde{p}$:

$$H(p)(\tilde{p}) = \frac{d}{d\epsilon}\left(\frac{d}{dp}J(u(p+\epsilon\tilde{p}),p+\epsilon\tilde{p})\right)\Bigg|_{\epsilon=0}. \tag{18}$$

The finite dimensional representation of the Hessian action is given by:

$$H(p)(\tilde{p}) = \frac{d}{dp}\left(\frac{d}{dp}J(u(p),p)\right)\tilde{p}. \tag{19}$$

. Considering directions $\tilde{u}$ and $\tilde{\lambda}$ associated with $\tilde{p}$, we can expand this using the chain rule:

$$H(p)(\tilde{p}) = \frac{\partial}{\partial p}\left(\frac{d}{dp}J(u,p)\right)\tilde{p} + \frac{\partial}{\partial u}\left(\frac{d}{dp}J(u,p)\right)\tilde{u} + \frac{\partial}{\partial \lambda}\left(\frac{d}{dp}J(u,p)\right)\tilde{\lambda}. \tag{20}$$

Using our previous expression for the gradient, we can write the Hessian action as:

$$\begin{aligned} H(p)(\tilde{p}) =& \frac{\partial}{\partial p}\left(\frac{\partial J}{\partial p}(u,p)\right)\tilde{p} + \frac{\partial}{\partial u}\left(\frac{\partial J}{\partial p}(u,p)\right)\tilde{u} \\ & - \tilde{\lambda}^{T}\frac{\partial F}{\partial p}(u,p) - \lambda^{T}\frac{\partial}{\partial p}\left(\frac{\partial F}{\partial p}(u,p)\right)\tilde{p} - \lambda^{T}\frac{\partial}{\partial u}\left(\frac{\partial F}{\partial p}(u,p)\right)\tilde{u}. \end{aligned} \tag{21}$$

To compute this, we need to determine $\tilde{u}$ and $\tilde{\lambda}$. We find $\tilde{u}$ by differentiating the pde constraint $F(u,p)=0$ with respect to p in the direction $\tilde{p}$:

$$\frac{\partial F}{\partial u}(u,p)\tilde{u}+\frac{\partial F}{\partial p}(u,p)\tilde{p}=0, \tag{22}$$

which requires solving the **incremental state equation**:

$$\left(\frac{\partial F}{\partial u}(u,p)\right)\tilde{u}=-\frac{\partial F}{\partial p}(u,p)\tilde{p}. \tag{23}$$

Next, we find $\tilde{\lambda}$ by differentiating the adjoint equation with respect to p in the direction $\tilde{p}$:

$$\begin{aligned}&\left(\frac{\partial F}{\partial u}(u,p)\right)^T\tilde{\lambda}+\left(\frac{\partial}{\partial p}\left(\frac{\partial F}{\partial u}(u,p)\right)\tilde{p}\right)^T\lambda+\left(\frac{\partial}{\partial u}\left(\frac{\partial F}{\partial u}(u,p)\right)\tilde{u}\right)^T\lambda\\&=\left(\frac{\partial}{\partial p}\left(\frac{\partial J}{\partial u}(u,p)\right)\tilde{p}\right)^T+\left(\frac{\partial}{\partial u}\left(\frac{\partial J}{\partial u}(u,p)\right)\tilde{u}\right)^T.\end{aligned} \tag{24}$$

Which gives the **incremental adjoint equation** to solve for $\tilde{\lambda}$:

$$\begin{aligned}\left(\frac{\partial F}{\partial u}(u,p)\right)^T\tilde{\lambda}=&\left(\frac{\partial}{\partial p}\left(\frac{\partial J}{\partial u}(u,p)\right)\tilde{p}\right)^T+\left(\frac{\partial}{\partial u}\left(\frac{\partial J}{\partial u}(u,p)\right)\tilde{u}\right)^T\\&-\left(\frac{\partial}{\partial p}\left(\frac{\partial F}{\partial u}(u,p)\right)\tilde{p}\right)^T\lambda-\left(\frac{\partial}{\partial u}\left(\frac{\partial F}{\partial u}(u,p)\right)\tilde{u}\right)^T\lambda.\end{aligned} \tag{25}$$